Lctr 3: MCU Architecture, Instruction Pipeline, Memory, and Number Expression

Jianhua Liu

Spring 2026

==Topics==

Core concepts and applications:

- Architecture of ARM Cortex-M processors.
- Pipelines for instruction execution.
- Memory organization.
- Expressions of unsigned integers saved in memory: decimal, binary, and hexadecimal formats.

Demo project:

- Printing numbers in different formats with UART redirect.
- Reading numbers with UART redirect with certain range.

After-class review by reading Sec 3.6 of the Collection:

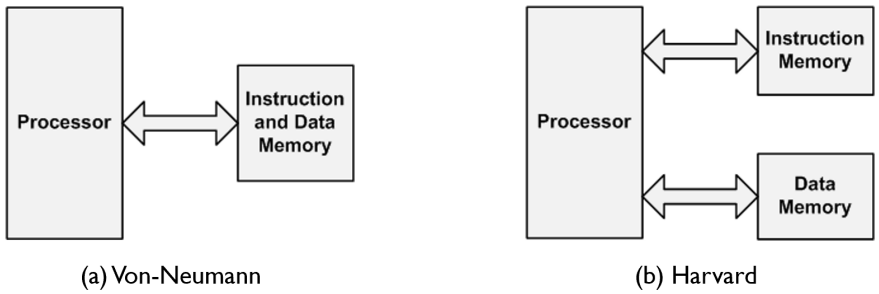
- C programming:
  - Static and global functions.
  - Static and global variables.
  - `for` and `while` loops.
  - `char` and ASCII coding.
  - Strings and their ending.

3.1 Introduction

3.2 Architecture of Cortex-M processors and registers

3.2.1 Architecture of Cortex-M processors

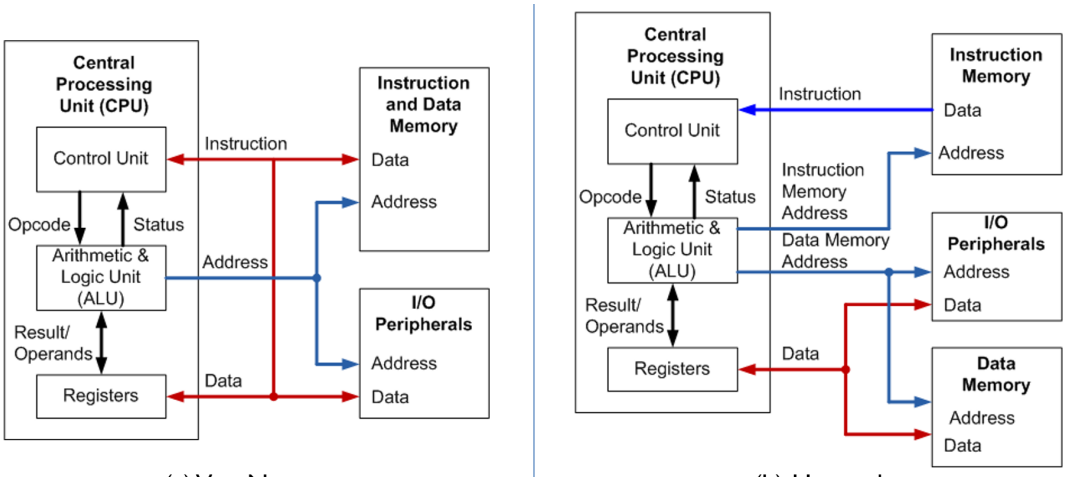
Cortex-M processors use two different architecture: Von-Neumann for Cortex-M0/0+ and Harvard for Cortex-M3/4/7. A simplified version of these two architectures are shown in Fig. 1.



(a) Von-Neumann (b) Harvard

Figure 1: STM32H7A3 and STM32H7B3 MCUs with different pin counts and memory sizes.

More detailed illustrations of these two architectures are shown in Fig. 2.



(a) Von-Neumann (b) Harvard

Figure 2: STM32H7A3 and STM32H7B3 MCUs with different pin counts and memory sizes.

Note that Cortex-M processors are 32-bit processors, which means that all the general-purpose registers used for operations are 32-bit wide. For normal operations, the ALU is also 32-bit wide. Yet, we can perform multiplications of two 32-bit numbers to obtain a 64-bit result.

The width of the address bus is 32-bit as well.

3.2.2 Processor registers

Cortex-M processors have a number of processor registers, as shown in Fig. 3

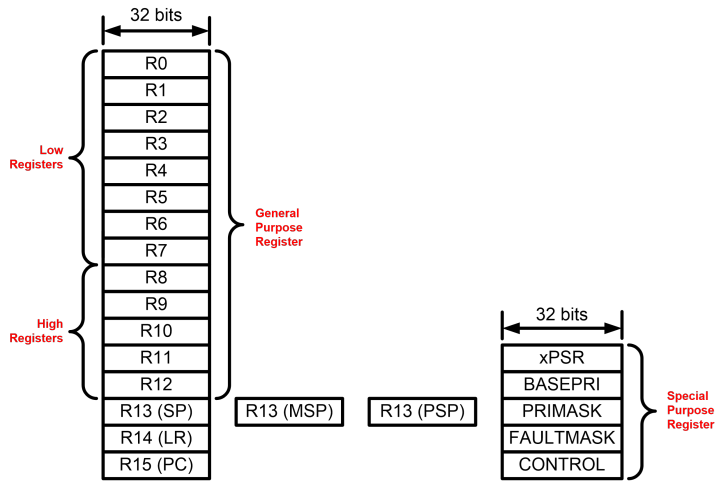


Figure 3: STM32H7A3 and STM32H7B3 MCUs with different pin counts and memory sizes.

3.3 Memory space

3.3.1 Memory organization

Although the processor core is 32-bit wide, the data and instructions in the memory are saved in units of **bytes**, which are only 8-bit wide, in the memory space, as shown in Fig. 4.

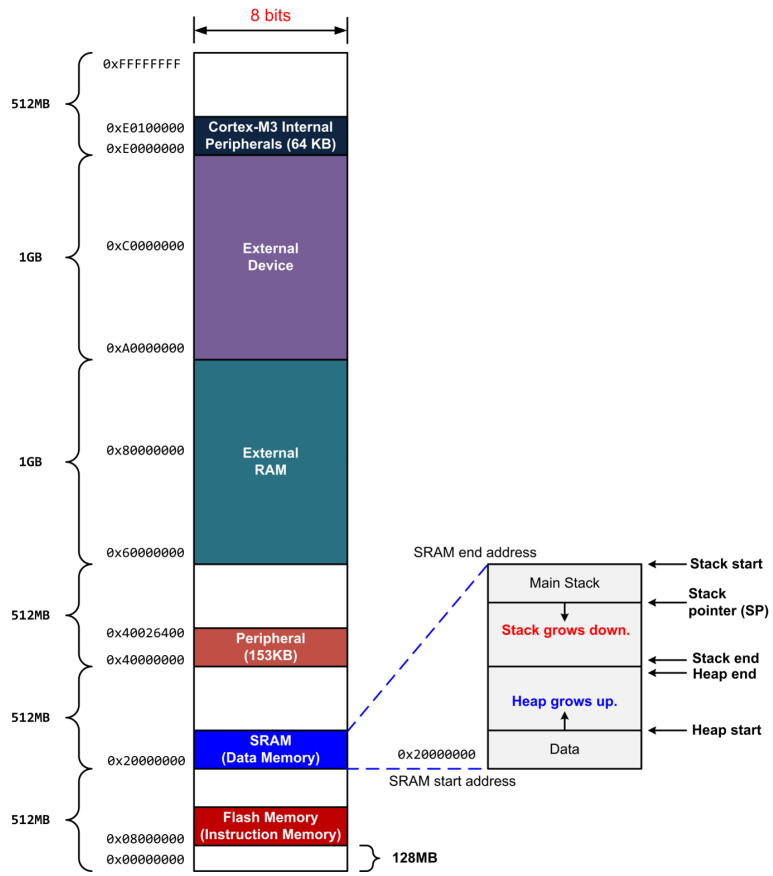


Figure 4: STM32H7A3 and STM32H7B3 MCUs with different pin counts and memory sizes.

We have a number of remarks regarding the memory organization.

- **The minimum unit of storage is byte** so each byte is **addressed** uniquely.
- A 32-bit address bus can address $2^{32} = 4 \times 2^{30}$ bytes = 4 GiB (gibibyte).
  - Later, we will just call it as 4 GB for simplicity.
  - The meaning of 2^{32} will be clear after we review/discuss binary expression below.
- We can combine two bytes to a 16-bit number, which is called a **halfword**.
- We can combine two halfwords to a 32-bit number, which is called a **word**.
- There are a lot of data that are only 8-bit wide, as we have seen in the UART communications. Using byte-based, instead of word-based, addressing can save a lot of memory spaces when we deal with bytes (or halfwords) often.
- Although the memory is byte-based, we can read a 32-bit word, altogether 4 bytes, in the same time cycle used for reading a single byte if the 4 bytes are aligned to the boundary of multiples of 4 bytes. This will be discussed in more detail later.
- Peripherals are controlled using **peripheral registers** that are accessed in the same way as for data memory. This greatly simplifies the instruction design.
- Although the Harvard architecture supports separate busses for data and instructions, these memory blocks do not use the same addresses (which is affordable as there is so much space). This leads to the benefit that the address assignments for different memory or peripherals can be consistent between processors of different families.

3.3.2 Flash vs RAM

There are two types of memories in the MCU, flash and RAM (random access memory). Roughly speaking:

- Flash is used to save instruction code.
- RAM is used to save data.

3.4 Instruction pipeline

Cortex-M processors use instruction pipelines to improve the efficiency of instruction execution. Fig. 5 shows a 3-stage pipeline.

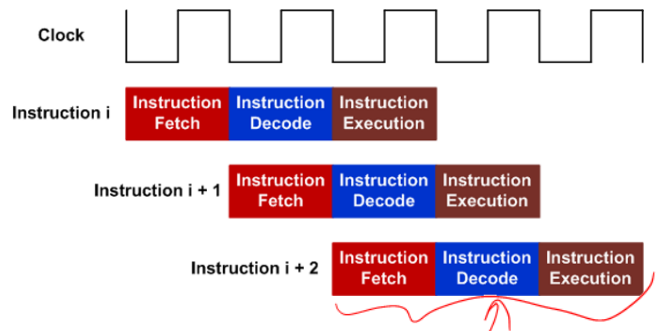


Figure 5: STM32H7A3 and STM32H7B3 MCUs with different pin counts and memory sizes.

For a processor with a **3-stage** pipeline, it takes **3 clock cycles** to execute a single instruction. Yet, when we put n instructions in the pipeline, we will use $n + (3 - 1) = n + 2$ total clock cycles. This will be able to reduce the time used to execute each instruction on average, which corresponds to an improved **throughput**, which is defined as *the total number of real instructions of a block of code over the total numbers of clock cycles of the code*.

Example 1: Consider a Cortex-M4 processor which has a 3-stage instruction pipeline running code in a loop. Assume:

- There are 4 instructions to run the basic task in a loop, say copying data from one memory location to another.
- There is one additional branch instruction at the end of the loop go back to the beginning of the loop to form the loop.
- When the branch instruction is executed, the pipeline is flushed, and we need to start over the pipeline.

Question:

1. What is the throughput of this loop?
2. If we modify the loop to repeat the 4 instructions 5 times in the loop, what is the throughput of the new longer loop?

Solution:

1. Including the branch instruction, there are 4 + 1 = 5 instructions in the loop, as shown in the ASM pseudo below. So the total number of clock cycles for running the loop is 5+2. Hence, the throughput is (5 Insts)/(7 cycs) = $\frac{5}{7}$ Insts/c

   loop: Inst 1, Inst 2, Inst 3, Inst 4 } CB
   Branch loop

2. If we modify the loop to repeat the 4 instructions 5 times in the loop, as shown in the ASM pseudo below, there will be 4×5 = 20 instructions for the basic task and 1 instruction for the branch. Altogether, we have 21 instructions, which takes 23 clock cycles. Then the throughput is (21 Insts)/(23 cycs) = $\frac{21}{23}$ Insts/c

   loop: CB 1
   CB 2
   CB 3
   CB 4
   CB 5
   Branch loop

-:-

3.5 Number expression and the demo project

All numbers in a computer system are saved in the **binary** format. If a number is a nonnegative integer, we call it **unsigned**. Unsigned integers have the easiest mapping between the binary data saved on the computer and the value of the number that we want the binary data to *represent*.

The demo project, `ce1s_print_in_diff_formats`, can be used to demonstrate the printing of numbers in different format. The code will be illustrated below during the review of the number expressions.

3.5.1 Four different expressions of unsigned integers

We have mainly **four** ways to express the value of an unsigned integer encoded in the binary data, as shown in the Table below.

Decimal	Binary	Octal	Hex	Decimal	Binary	Octal	Hex
0	0b0000	00	0x0	8	0b1000	010	0x8
1	0b0001	01	0x1	9	0b1001	011	0x9
2	0b0010	02	0x2	10	0b1010	012	0xA
3	0b0011	03	0x3	11	0b1011	013	0xB
4	0b0100	04	0x4	12	0b1100	014	0xC
5	0b0101	05	0x5	13	0b1101	015	0xD
6	0b0110	06	0x6	14	0b1110	016	0xE
7	0b0111	07	0x7	15	0b1111	017	0xF

How to signify different expressions:

- `70`: decimal as there is no prefix.
- `0b1000110`: binary as there is a `0b` prefix.
- `0106`: octal as there is a `0` prefix.

- `0x46`: hexadecimal as there is a `0x` (or `0X`) prefix.

To facilitate the reading of the number, we can use an underscore to separate the bits or digits into groups of 4, starting from the least significant bit (LSB) or digit (LSD) . By using this approach, we can write the above `0b1000110` as `0b100_0110`. We can also write it as `0b0100_0110` to make it explicitly 8 bit. This notation can be used for hexadecimals as well.

Consider decimal num:
123
$= 1\times10^2 + 2\times10^1 + 3\times10^0$

3.5.2 Conversion from binary, octal, or hexadecimal to decimal

This conversion is easy; just use the following equation

$$(d_3d_2d_1d_0)_r = d_3 \times r^3 + d_2 \times r^2 + d_1 \times r^1 + d_0 \times r^0,$$

where r is the radix, which is 2, 8, and 16 for binary, octal, and hexadecimal, respectively. In the above equation, d_0 is the LSB for binary and the LSD for other expressions.

3.5.3 Conversion from decimal to binary using the divide-by-2 method

We can use the above principle of *conversion from any format to decimal* to see how to do the conversion from decimal to binary. Consider

$$\begin{aligned} A_0 = (d_3d_2d_1d_0)_r &= (d_3 \times r^3 + d_2 \times r^2 + d_1 \times r^1) + d_0 \\ &= (\underbrace{d_3 \times r^2 + d_2 \times r^1 + d_1}_{A_1}) \times r^1 + d_0 \end{aligned}$$

Now, if we write $A_0 = A_1 \times r + d_0$, we know that $d_0 = A_0 \% r$ (here $\%$ is the modulo operator which finds the remainder) and $A_1 = A_0/r$ (integer division with the result being an integer). This way, we obtained d_0 and A_1. Similarly, we can obtain $d_1 = A_1 \% r$ and $A_2 = A_1/r$. Keep doing this, we can obtain $d_{n-1} = A_{n-1} \% r$ and $A_n = A_{n-1}/r = 0$. We stop when A_n is zero.

Now, let us illustrate this using $A_0 = 13$ and binary expression (so we use `b` instead of `d`).

Example 2: Convert 13 into binary. Determine the values of the 8 LSBs. Also, place these bits into `str`, a string of length 8. (Note that we need an array of 9 bytes for this array as the last element should be the **null terminator**, a true `0`.)

$A_0 = 13 = 6 \times 2 + 1$

Solution:

$$b_0 = A_0 \% 2 = 1, \qquad A_1 = 13/2 = 6; = 3 \times 2 + 0$$

$$b_1 = A_1 \% 2 = 0, \qquad A_2 = 6/2 = 3;$$

$$b_2 = A_2 \% 2 = 1, \qquad A_3 = 3/2 = 1;$$

$$b_3 = A_3 \% 2 = 1, \qquad A_4 = 1/2 = 0,$$

$$b_4 = A_4 \% 2 = 0$$

str =	'0'	'0'	'0'	'0'	'1'	'1'	'0'	'1'	0
	0	1	2	3	4	5	6	7	8

Index of str

-:-

C implementation of the above algorithm is shown below.

```c
static void num_2_n_bit_bin_str_div_by_2(char *str, uint32_t num, int n) {
    num = mask_n_bits(num, n);
    int i = 0;   // index for the bit (or digit)
    while (num) {
        int b_i = num % 2;
        str[n - 1 - i++] = '0' + b_i;
        num = num / 2;
    }
    while (i < n) {
        str[n - 1 - i++] = '0';
    }
    str[n] = 0;  // end the string.
}
```

The `mask_n_bits` function is a helper function used to make the caller function clean.

```
static uint32_t mask_n_bits(uint32_t num, int n) {
    uint32_t n_bit_mask = (1U << n) - 1;     // 0b11 ... 11
    return num & n_bit_mask;                 //   \-- n --/
}
```

Here `1U` specifies that `1` is an *unsigned long* integer, which is guaranteed to be 32 bits. Other parts of the code, including the concept of mask, are explained below in the C programming notes section below.

Note that there are other approaches to do the above conversion, such as the following one which is based on a shift-to-right method.

```
static void num_2_n_bit_bin_str_shift2rt(char *str, uint32_t
    num, int n) {
    num = mask_n_bits(num, n);
    for (int i = 0; i < n; i++) {  // i for bit index
        int b_i = (num >> i) & 1U;
        *(str + n - 1 - i) = '0' + b_i;
    }
    *(str + n) = 0; // end the string.
}
```

To have a better understanding of the above code, please read the following in the Additional reading section of Chapter 3 of the Collection:

- Shift operations and creation of n 1's.
- Masking with a mask and bitwise AND operation.
- `for` and `while` loops.
- ASCII coding and the terminating of a string.
- Array vs pointer.

$i=0$:
$(num >> i) = 0b\ b_7 b_6 b_5 b_4_b_3 b_2 b_1 b_0$
& 1U $\quad 0b\ 0000_0001$
$\quad \ldots 0000\ b_0$

$i=1$:
$(num >> i) = 0b\ 0\ b_7 b_6 b_5_b_4 b_3 b_2 b_1$
& 1U $\quad 0000_0001$
$\quad 0000\ 0000\ b_1$

We will go over these topics systematically, but a reading now can help understanding the code well.

3.5.4 Unsigned conversion from binary to octal or hexadecimal

This can be done easily by regrouping the bits. We group them to 3-bit or 4-bit groups **starting from the LSB** to MSB for octal and hexadecimal, respectively. We will pad zeros on the MSB side if needed.

After class:

- Please read other topics of Additional reading section as much as possible.
- Try to relate the sizes of different memory blocks with their addresses as labeled in Figure 4.